Object Oriented Programming (CST-004)

Complete unit-wise notes - easy English, matched to the UTU syllabus.

Study Materials by Anas - study-materials-by-anas.vercel.app

Unit 1 - OOP Concepts, Classes & Pointers

What this unit covers: the four pillars of OOP, how C++ classes and objects work, and pointers - the foundation for everything after.

Key ideas: Encapsulation = data and functions bundled in one class; **abstraction** = show what, hide how; **inheritance** = reuse by deriving classes; **polymorphism** = same name, many behaviours. A **class** is a blueprint, an **object** its instance. Access specifiers: **private** (default - only inside the class), **public** (anywhere), **protected** (class + derived classes). A **pointer** stores an address: * declares/dereferences, & takes an address. **new/delete** manage heap memory, the **this** pointer points to the calling object. A **reference** is an alias - must be initialised and cannot be reseated.

Exam pointers: "explain the four pillars with examples" is the classic theory question. Expect small code outputs using pointers (* and &) and a class with private data accessed through public member functions. Know the default specifier: class = private, struct = public.

Unit 2 - Function Overloading, Constructors & Operator Overloading

What this unit covers: function overloading, constructors and destructors, and operator overloading - making classes behave like built-in types.

Key ideas: Function overloading: same name, different parameter lists - the compiler picks by the arguments. **Constructors** run automatically at object creation: default, parameterised, and **copy constructor** (Class(const Class&) - initialises a new object from an existing one; know when it is called). The **destructor** (~Class()) runs when the object dies, in reverse order of construction. **Operator overloading** gives operators a meaning for your class, e.g. Complex operator+(Complex&). Remember: you cannot overload ::, .*, ., ?:, sizeof; and operators like =, (), [] must be member functions. A **friend function** is not a member but can touch private data - declare it inside the class with the friend keyword.

Exam pointers: writing operator+ for a Complex class (and operator++ pre/post) is the standard program. "Order of constructor/destructor calls" output questions and "difference between constructor and destructor" appear regularly.

Unit 3 - Inheritance & Polymorphism

What this unit covers: inheritance and polymorphism - deriving new classes from old ones and making the right function run at runtime.

Key ideas: Inheritance lets a derived class reuse a base class. Types: **single** (one base, one derived), **multilevel** (A → B → C), **multiple** (many bases), **hierarchical** (one base, many derived), **hybrid** (a mix). Access control: public inheritance keeps public members public and protected members protected; private inheritance makes everything private in the derived class. **Polymorphism:** declare a base function **virtual** and a base pointer will call the derived version at runtime (dynamic binding). A **pure virtual** function (= 0) makes the class **abstract** - it cannot be instantiated. In **multiple inheritance** the diamond problem (two copies of a grandparent) is fixed with a **virtual base class**.

Exam pointers: output prediction with virtual vs non-virtual functions is the repeated question; also "types of inheritance with diagrams" and "what is an abstract class / virtual base class". Constructor order in inheritance (base first, then derived) is a favourite 2-marker.

Extra note - RTTI & Casting - made simple:

RTTI (Run-Time Type Identification) = finding out, **while the program is running**, what type an object actually is.

> *Imagine a box labelled "fruit" - inside could be an apple or a mango. RTTI is opening the box to see what is really inside.*

- **typeid(object)** - tells the object's type. Example: `typeid(x).name()`
- **dynamic_cast** - safely converts a base pointer to a derived one. On a wrong conversion it returns **nullptr** (no crash).
- **static_cast** - normal conversion at compile time (like int to float).
- **const_cast** - adds or removes const. **reinterpret_cast** - risky; just re-reads the bits.

Exam tip: dynamic_cast works only when the base class has at least one **virtual function** (a polymorphic class).

Unit 4 - Templates & Exception Handling

What this unit covers: templates for generic code and exception handling for runtime errors.

Key ideas: A **function template** is one function that works for any type: template <class T> T add(T a, T b). A **class template** generalises a whole class (like a Stack<T> for int or string). The compiler generates a concrete version for each type you use - this is compile-time (static) polymorphism. **Exception handling:** code that may fail goes in **try**, an error is signalled with **throw**, and a matching **catch** block handles it. catch(...) catches anything. If no handler matches, the program terminates. You can rethrow from a catch and write your own exception classes.

Exam pointers: expect one program using a function template (max/swap/sort for two types) and one using try/throw/catch - divide-by-zero is the usual example. Theory: "templates vs macros" and "why exceptions beat error codes" are common short answers.

Unit 5 - File I/O, Namespaces & STL

What this unit covers: file input/output, namespaces, and a first look at the STL.

Key ideas: File I/O uses streams: **ifstream** (read), **ofstream** (write), **fstream** (both). Open with a mode - ios::in, ios::out, ios::app (append), ios::binary - check is_open(), use << / >> or get()/put(), getline() for whole lines, then close(). **Namespaces** group names to avoid clashes: namespace std is why we write std::cout or add using namespace std. **STL** (Standard Template Library) gives ready containers - **vector** (dynamic array), **list** (doubly linked list), **map** (key → value, sorted), **set** (unique sorted values) - plus **iterators** to walk them and **algorithms** like sort() and find().

Exam pointers: a program that writes data to a file and reads it back is the standard question - practise it with getline. Short theory: file opening modes table, "what is a namespace and why", and "name three STL containers with one use each".

Made by Anas. Free forever. Watch the linked videos on the site after reading each unit.